Big Data Analytics

Dr Sirintra Vaiwsri | Email: sirintra.v@itm.kmutnb.ac.th

Batch Data Processing

Batch Processing (Antolínez García, 2023)

- Batch processing is the computing method for executing large amounts of batch data.
- The batch process does not require an interaction with the user once the process begins.
- It is suitable for end-of-cycle treatment, such as updating stock at the end of a day.
- However, when any error occurs, the batch process can be shut down.

Spark Application

Spark Application (Antolínez García, 2023)

- Spark Application is a program developed by users over the Spark using Spark APIs.
- SparkSession communicates Spark with users via Spark APIs.
- Task is the smallest execution unit that is executed in an executor.
- Stage is a collection of tasks that execute the same code on different chunks of the dataset.
- Job consists of several stages and permits to execute the application in Spark.

Spark Application (Antolínez García, 2023)

- SparkSession is used in Spark 2.0 where SparkContext is used in the previous version of Spark.
- SparkSession is created using the builder function:

  `SparkSession.builder()`

- To retrieve an existing session, use the function `getOrCreate()`.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()
print(spark)
```

```
<pyspark.sql.session.SparkSession object at 0x7fd9d0ccc3d0>
```

Spark Application (Antolínez García, 2023)

- Spark operations are classified into 2 operations:
  - Transformations
  - Actions

Transformations (Antolínez García, 2023)

- Transformations take a Resilient Distributed Dataset (RDD) or DataFrame as an input and return RDD or DataFrame as an output.
- Characteristics:
  - Immutable - preserves the original copy of data
  - Not executed immediately - memorised and creates a transformation lineage which is a sequence of operations recorded in a Directed Acyclic Graph (DAG) that will be executed when an action is called. This is known as Lazy Evaluation.

Transformations

(Antolínez García, 2023; Chambers and Zaharia, 2018)

- Two types of transformations:
  - Narrow transformations - operations without data shuffling, in other words, one data partition results in one output partition.
    - map(), mapPartition(), filter(), union()
  - Wide transformations - operations involve data shuffling, in other words, one data partition results in multiple output partitions.
    - join(), distinct(), aggregate(), repartition(), intersect()

Actions (Antolínez García, 2023)

- Spark operations return a single value.
- It triggers the transformations.
- Example functions: collect(), count(), min(), max(), top(), etc.

Resilient Distributed Datasets (RDDs)

RDDs (Antolínez García, 2023)

- Datasets are divided into logical partitions.
- These partitions are processed in parallel across different nodes.
- RDDs can be created by existing collections or from external datasets such as text, CSV, and JSON files.

RDDs from Parallelised Collections

(Antolínez García, 2023)

- A collection of elements are transformed into distributed datasets.
- These datasets are then processed in parallel.
- RDDs created from a collection of elements using `sparkContext.parallelize()`
- Number of partitions is automatically defined based on the number of nodes available.
- However, the number of partitions can be user defined.

RDDs from Parallelised Collections

(Antolínez García, 2023)

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()
alphabet = [‘a’, ‘b’, ‘c’, ‘d’, ‘e’, ‘f’, ‘g’, ‘h’]
rdd = spark.sparkContext.parallelize(alphabet, 4)
print(“Number of partitions: ” + str(rdd.getNumPartitions()))
```

Use textFile() to create RDD from file.

```
rdd2 = spark.sparkContext.textFile(‘fb_live_thailand.csv’, 5)
print(“Number of partitions: ” + str(rdd2.getNumPartitions()))
```

RDDs from Parallelised Collections

(Antolínez García, 2023; Chambers and Zaharia, 2018)

- textFile() returns a record per line
- wholeTextFiles() returns the path to the file and file content in the form of a key-value pair where each file becomes a single record.
- However, wholeTextFiles() prefers small file processing.

Transformation Functions (Chambers and Zaharia, 2018)

- Distinct is used to remove duplicate records from the RDD

```
count_distinct = rdd.distinct().count()
print("Number of distinct records: ", count_distinct)
```

- Filter is used to select records that match the defined condition

```
filter_rdd = rdd.filter(lambda x: x.split(',')[1] == 'link').collect()
print(filter_rdd)
```

Transformation Functions

(Antolínez García, 2023; Chambers and Zaharia, 2018)

- Map is used to create key-value pairs.
- flatMap provides an extension of the Map.
- flatMap can be used when each input should return multiple outputs.

```
flatmap = rdd.flatMap(lambda x: x.split(‘,’))
pair = flatmap.map(lambda x: (x,1))
take_pair = pair.take(200)
for f in take_pair:
        if str(f[0]) == ‘photo’ or str(f[0]) == ‘video’:
                print(str(f[0]), str(f[1]))
```

Transformation Functions (Antolínez García, 2023)

- Sort is used to sort values in the RDD.
- sortByKey() is used for sorting a pair RDD based on Key.

```
sort_data = pair.sortByKey().collect()
for f in sort_data:
        print(str(f[0]), str(f[1]))
```

Transformation Functions (Antolínez García, 2023)

- sortBy() is another sort function.
- It receives 3 arguments which are key, flag (True/False) for ascending or descending order, and number of partitions.

```
sort_data = pair.sortBy(lambda x:x, False, 5).collect()
for f in sort_data:
        print(str(f[0]), str(f[1]))
```

Transformation Functions (Antolínez García, 2023)

- reduceByKey() merges the values of each key using the reduce function.
- It is a wide transformation that shuffles data across all partitions.

```
reduce_key = pair.reduceByKey(lambda x,y: x+y))
print(reduce_key.take(10))
```

Transformation Functions (Antolínez García, 2023)

- aggregateByKey() combines values based on the key at the partition level.
- Outputs from partitions are then combined across nodes.

```
alphabet = [('a',1), ('b',2), ('c',3), ('a',1), ('b',2)]
rdd = spark.sparkContext.parallelize(alphabet, 4)
zero_val = (0,0)
par_agg = lambda x,y: (x[0] + y, x[1] + 1)
allpar_agg = lambda x,y: (x[0] + y[0], x[1] + y[1])
agg = rdd.aggregateByKey(zero_val, par_agg, allpar_agg).take(10)
print(agg)
```

Transformation Functions (Antolínez García, 2023)

- foldByKey() combines values based on Key.
- It initialises value per key and partition where the initialised value will not affect the final result.

```
from operator import add
alphabet = [('a',1), ('b',2), ('c',3), ('a',1), ('b',2)]
rdd = spark.sparkContext.parallelize(alphabet, 4)
fold = sorted(rdd.foldByKey(0, add).collect())
print(fold)
```

Transformation Functions (Antolínez García, 2023)

- combineByKey() requires a shuffle in the last stage.
- It transforms key-value pairs into key-combined values.

```
alphabet = [('a',1), ('b',2), ('c',3), ('a',1), ('b',2)]
rdd = spark.sparkContext.parallelize(alphabet, 4)
def tolist(x):
        return [x]
def append(x, y):
        x.append(y)
        return x
def extend(x, y):
        x.extend(y)
        return x
combine = sorted(rdd.combineByKey(tolist, append, extend).take(10))
print(combine)
```

Transformation Functions (Antolínez García, 2023)

- groupByKey() groups values based on key.
- The key-value pairs are shuffled across all nodes.
- Therefore, it is not suitable for huge datasets.

```
alphabet = [('a',1), ('b',2), ('c',3), ('a',1), ('b',2)]
rdd = spark.sparkContext.parallelize(alphabet, 4)
group1 = sorted(rdd.groupByKey().mapValues(len).take(10))
print(group1)
group2 = sorted(rdd.groupByKey().mapValues(list).take(10))
print(group2)
```

Transformation Functions (Antolínez García, 2023)

- join() returns the pairs with the keys that are common between pairs.
- leftOuterJoin() returns the pair with the keys that are in the left RDD.
- rightOuterJoin() returns the pair with the keys that are in the right RDD.

```
alphabet1 = [('a',1), ('b',2), ('c',3)]
rdd1 = spark.sparkContext.parallelize(alphabet1)
alphabet2 = [('a',1), ('b',2), ('a',1), ('b',2)]
rdd2 = spark.sparkContext.parallelize(alphabet2)
joinRDD = rdd1.join(rdd2).collect()
print(joinRDD)
left = rdd1.leftOuterJoin(rdd2).collect()
print(left)
right = rdd1.rightOuterJoin(rdd2).collect()
print(right)
```

Action Functions (Antolínez García, 2023)

- countByKey() counts the values for each key.
- It returns a DefaultDict which is a dictionary object.

```
alphabet = [(‘a’,1), (‘b’,2), (‘c’,3), (‘a’,1), (‘b’,2)]
rdd = spark.sparkContext.parallelize(alphabet)
count = rdd.countByKey()
print(count)
```

Action Functions (Antolínez García, 2023)

- countByValue() counts unique values and returns a dictionary of value-count pairs.

```
alphabet = [('a',1), ('b',2), ('c',3), ('a',1), ('b',2)]
rdd = spark.sparkContext.parallelize(alphabet)
count_val = rdd.countByValue()
print(count_val)
```

Action Functions (Antolínez García, 2023)

- collectAsMap() returns output as a dictionary

```
alphabet = [('a',1), ('b',2), ('c',3), ('a',1), ('b',2)]
rdd = spark.sparkContext.parallelize(alphabet)
col_asmap = rdd.collectAsMap()
print(col_asmap)
```

Action Functions (Antolínez García, 2023)

- lookup(key) returns all values based on the key in the form of a list.

```
alphabet = [('a',1), ('b',2), ('c',3), ('a',1), ('b',2)]
rdd = spark.sparkContext.parallelize(alphabet)
look = rdd.lookup('a')
print(look)
```

Action Functions (Chambers and Zaharia, 2018)

- first() returns the first value in the dataset.

```
alphabet = [('a',1), ('b',2), ('c',3), ('a',1), ('b',2)]
rdd = spark.sparkContext.parallelize(alphabet)
first = rdd.first()
print(first)
```

Action Functions (Chambers and Zaharia, 2018)

- max() and min() returns maximum and minimum values in the dataset, respectively.

```
alphabet = [('a',1), ('b',2), ('c',3), ('a',1), ('b',2)]
rdd = spark.sparkContext.parallelize(alphabet)
max = rdd.max()
print(max)
min = rdd.min()
print(min)
```

Action Functions (Antolínez García, 2023)

- saveAsTextFile() stores RDD as a text file.

```
import tempfile
alphabet = [('a',1), ('b',2), ('c',3), ('a',1), ('b',2)]
rdd = spark.sparkContext.parallelize(alphabet)
with tempfile.TemporaryDictionary() as data:
    folder = "textfile"
    rdd.saveAsTextFile(folder)
```

Assignment (1 point)

- Please implement and show the results of all transformation and action functions to get 1 point.

References

- Antolínez García, A. (2023). *Hands-on Guide to Apache Spark 3: Build Scalable Computing Engines for Batch and Stream Data Processing*. Berkeley, CA: Apress.
- Chambers, B., & Zaharia, M. (2018). *Spark: The definitive guide: Big data processing made simple*. " O'Reilly Media, Inc.".